❶ Sink for invalid Result elements

❷ Sink for valid Result elements

❸ More processing (ignored here)

One of the advantages of `divertTo` is that once you know you can divert to another sink, you don't have to perform any further operations on that element. The stream becomes more efficient if you send it to its final destination as quickly as possible. The sinks in the example only print, but with everything you know by now, you can easily perform an upsert against a database, send to an HTTP service, or write to a Kafka topic.

Exceptions sometimes occur not within a stream but rather at its source. That is another problem.

## *16.3.4 Restarting the source*

One of the most common scenarios when working with streams is creating the source from an external service, an HTTP service, or a Kafka topic. But if this external source is unavailable for some time, your stream will fail.

So far, you haven't seen a way to automatically restart a source. Let's lay the groundwork. The plan is to use a gRPC service from a source so that even if the service shuts down or crashes, the stream can continue once the service comes back online. The following service is in the chapter16 project in the locator.proto file.

**Listing 16.8 `LocatorService` locator.proto file**

```
import "google/protobuf/empty.proto";                    ❶

message Location {                                       ❷
```